Fiona Mobile Operations PWA

Mobile-first εφαρμογή PWA για την καθημερινή διαχείριση εισιτηρίων Fiona. Περιλαμβάνει React/Vite frontend και backend Node.js/Express που κάνει polling στα δεδομένα της Fiona.

Οδηγός χρήσης

Η εφαρμογή εμφανίζει τα ανοιχτά εισιτήρια, την κατάστασή τους, τα στοιχεία εργασίας, τις προθεσμίες και τα στοιχεία επικοινωνίας. Από τον επιλογέα καταστήματος μπορείς να δεις ένα κατάστημα ή την ενιαία λίστα όλων των ρυθμισμένων καταστημάτων.

1. Άνοιξε τη διεύθυνση της εφαρμογής και συνδέσου με τον κωδικό εφαρμογής που σου έχει δοθεί. Δεν χρειάζεσαι τον κωδικό Fiona.
2. Χρησιμοποίησε τα φίλτρα κατάστασης, την αναζήτηση και, στα ραντεβού, το φίλτρο συνεργείου.
3. Σε κάθε εισιτήριο μπορείς να δεις την προθεσμία, να καλέσεις τον πελάτη ή να ανοίξεις τη διεύθυνση στον χάρτη. Πάτησε «Λεπτομέρειες εισιτηρίου» για πρόσθετες πληροφορίες.
4. Το κουμπί ανανέωσης ζητά άμεσα τα νεότερα δεδομένα. Η λίστα ανανεώνεται αυτόματα περίπου κάθε λεπτό.
5. Στο φίλτρο `COMPLETE` άνοιξε τα «Ολοκληρωμένα» ή τα «Στατιστικά». Επίλεξε μήνα και πάτησε μια εργασία για τα πλήρη στοιχεία και τις διαθέσιμες ενέργειες αντιγραφής.

Η ένδειξη `Live` σημαίνει ότι ο server έχει πρόσφατα δεδομένα από τη Fiona. Οι ενδείξεις `Stale` ή `Offline / Cached` σημαίνουν ότι μπορεί να βλέπεις παλαιότερα δεδομένα. Περίμενε επιτυχημένο συγχρονισμό ή επιβεβαίωσε την κατάσταση στη Fiona πριν ενεργήσεις.

Fiona Team Points

Το ξεχωριστό tracker στο `/tracker/` δημιουργεί προφίλ ανά συνεργείο και κατάστημα Fiona και εμφανίζει τις ολοκληρωμένες εργασίες σε μηνιαίο ημερολόγιο. Οι πόντοι αποδίδονται από τον κατάλογο κατηγοριών του tracker. Δεν προστίθενται χειροκίνητες εργασίες.

Η εισαγωγή ξεκινά αυτόματα στην πρώτη εκκίνηση του backend με την έκδοση του tracker και δεν κάνει ιστορικό backfill. Σαρώνει τις πρόσφατες ημερομηνίες εργασιών, αλλά εισάγει μόνο orders με `orderStatus=COMPLETED`, έγκυρο `workOrderFinishedOnPremiseTime`, συνεργείο και ώρα ολοκλήρωσης ίση ή μεταγενέστερη από το cutover. Οι επαναλαμβανόμενες σαρώσεις δεν διπλομετρούν ένα order· η ταυτότητα είναι `(store_id, order_id)`.

Η αντιστοίχιση των πεδίων κατηγορίας της Fiona γίνεται με ακριβείς τιμές. Με το `PROJECT_TRACKER_CATEGORY_MAP` αντιστοιχίζονται επαληθευμένα Fiona labels ή category IDs σε IDs του tracker, π.χ. `{"Fiber install":"ftth_installation"}`. Εργασίες που δεν αντιστοιχίζονται εμφανίζονται ως εκκρεμείς χωρίς πόντους. Το συνεργείο μπορεί να διορθώσει την κατηγορία ή να ορίσει override πόντων με τον κωδικό του. Οι διορθώσεις κρατούνται σε audit log. Τα tracker API responses δεν περιλαμβάνουν ονόματα, τηλέφωνα, διευθύνσεις ή Α.Φ.Μ. πελατών.

Ανάπτυξη

Ροή εργασίας Git

Το `main` είναι η σταθερή branch. Όλες οι αλλαγές περνούν από Pull Request· μην κάνεις απευθείας commit ή force push στο `main`.

1. Ξεκίνα από ενημερωμένο `main` και δημιούργησε branch για μία συγκεκριμένη εργασία:

```
powershell git switch main git pull git switch -c feature/42-ticket-search
```

2. Χρησιμοποίησε ονόματα όπως `feature/<issue>-<περιγραφή>`, `fix/<issue>-<περιγραφή>` ή `hotfix/<περιγραφή>`. Αν δεν υπάρχει issue, παράλειψε τον αριθμό.
3. Κράτησε κάθε commit μικρό και σχετικό. Προτεινόμενα μηνύματα: `feat: add ticket search`, `fix: correct SLA calculation`, `docs: explain Git workflow`.
4. Στείλε το branch και άνοιξε Pull Request προς `main`:

```
powershell git push -u origin feature/42-ticket-search
```

5. Περίμενε να περάσουν οι έλεγχοι CI, έλεγξε το diff και κάνε merge από το GitHub. Διέγραψε το προσωρινό branch μετά το merge.

Το CI εκτελεί τα backend tests και τα builds του frontend και του tracker σε Pull Requests προς `main`. Το `development` δεν χρειάζεται ως μόνιμο branch για την τρέχουσα ροή· πρόσθεσέ το μόνο αν χρειάζεσαι ξεχωριστή staging έκδοση ή ομαδοποιημένες releases.

Για προστασία του `main`, ενεργοποίησε στο GitHub ruleset ή branch protection: απαίτηση Pull Request, απαίτηση επιτυχίας των `Backend tests`, `Frontend build` και `Tracker build`, απαγόρευση force push και διαγραφής branch. Αυτές οι ρυθμίσεις γίνονται χειροκίνητα στο GitHub και δεν ενεργοποιούνται από το workflow.

Δομή κώδικα

Το backend οργανώνεται σε επίπεδα με σαφή ευθύνη. Οι εξαρτήσεις πηγαίνουν προς μία κατεύθυνση: `routes` → `controllers` → `services` → `repositories`.

```
backend/src/
  server.js        εκκίνηση: listen, polling και tracker sync
  app.js           σύνθεση Express app, middleware και routes
  config.js        ρυθμίσεις από το περιβάλλον
  routes/          ορισμός endpoints (/api/tickets, /api/closed-tickets, /api/tracker, στατικά apps)
  controllers/     χειρισμός request/response, επικύρωση παραμέτρων
  middleware/      auth και session έλεγχοι
  services/        επιχειρησιακή λογική (Fiona client, polling, SLA, στατιστικά, sync)
  repositories/    πρόσβαση στη SQLite
  cli/             εντολές γραμμής εντολών (sync:closed)
```

```
frontend/src/
  main.jsx, App.jsx, styles.css
  components/      κοινά UI στοιχεία (Icon)
  shared/          βοηθητικές συναρτήσεις χωρίς εξάρτηση από feature
  features/
    auth/            AuthGate
    tickets/         κάρτα εισιτηρίου, μοντέλο/SLA, formatters, τοπική αποθήκευση, demo δεδομένα
    closed-tickets/  προβολή ολοκληρωμένων και στατιστικών
```

Νέο endpoint: πρόσθεσε controller στο `controllers/`, route στο `routes/` και σύνδεσέ το στο `app.js`. Νέα λειτουργία διεπαφής: δημιούργησε φάκελο στο `frontend/src/features/`.

Αρχιτεκτονική

- `frontend/`: React 18, Vite και PWA service worker στο `/app/`. Η ρίζα `/` ανακατευθύνει στο `/app/`, ενώ η διεπαφή ζητά δεδομένα από το ίδιο origin μέσω `/api/*`. Το production build βρίσκεται στο `frontend/dist`.
- `project-tracker/`: ξεχωριστό React/Vite PWA στο `/tracker/`, με δικό του manifest και service worker scope. Χρησιμοποιεί μόνο τα περιορισμένα `/api/tracker/*` endpoints.
- `backend/`: Node.js ES modules και Express. Κάθε ρυθμισμένο κατάστημα έχει ξεχωριστό Fiona client, session token και polling snapshot. Ο normalizer μετατρέπει κάθε απάντηση στο contract της διεπαφής και προσθέτει την ταυτότητα καταστήματος.
- Οι πίνακες `project_tracker_*` αποθηκεύουν μόνο order ID, κατάστημα, συνεργείο, κατηγορία, ώρα ολοκλήρωσης, πόντους/overrides και audit log. Χρησιμοποιούν την ίδια SQLite διαδρομή με τα closed tickets, αλλά ξεχωριστό schema.
- Ο Express server σερβίρει το frontend και το API από την ίδια θύρα. Η προεπιλεγμένη θύρα είναι `4000` και αλλάζει με το `PORT`.
- Το backend κρατά το πιο πρόσφατο snapshot στη μνήμη, χωριστά ανά κατάστημα. Το `POLL_INTERVAL_MS` ρυθμίζει το διάστημα polling: το `.env.example` ορίζει `300000` ms, ενώ ο fallback του κώδικα είναι `120000` ms. Η διεπαφή ζητά νέο snapshot περίπου κάθε 60 δευτερόλεπτα.
- Η SQLite βάση βρίσκεται από προεπιλογή στο `backend/data/closed-tickets.sqlite`. Οι πίνακες ολοκληρωμένων, εκκρεμών και παρατηρήσεων περιλαμβάνουν `store_id`, ώστε όμοια Fiona order IDs από διαφορετικά καταστήματα να μην συγκρούονται. Πριν από την πρώτη migration δημιουργείται δίπλα στη βάση το `closed-tickets.sqlite.pre-store-migration.bak` και οι παλιές εγγραφές μεταφέρονται στο `alexandroupoli`.
- Σε εντελώς κενό snapshot απαιτούνται δύο διαδοχικά επιτυχημένα polls, ώστε να αποκλειστεί προσωρινά κενή απάντηση. Το CLI `sync:closed` προορίζεται για ιστορική αναπλήρωση tickets που δεν είχαν καταγραφεί ως ανοιχτά πριν από shutdown.
- Για SLA ολοκλήρωσης συγκρίνεται το `workOrderFinishedOnPremiseTime` με το `workOrderSlaDate`. Το `deadline` είναι προθεσμία πρώτης κλήσης και συγκρίνεται με την πρώτη παρατήρηση `CALL LATER` ή `APPOINTMENT`, όχι με την ώρα ολοκλήρωσης.

Τοπική εκτέλεση

Από το project root εγκατέστησε τις εξαρτήσεις:

```
npm install --prefix backend
npm install --prefix frontend
npm install --prefix project-tracker
```

Δημιούργησε το `backend/.env` από το `backend/.env.example` και συμπλήρωσε το `FIONA_USERNAME` για Αλεξανδρούπολη, τα `FIONA_KOMOTINI_USERNAME` και `FIONA_ORESTIADA_USERNAME` για τα άλλα καταστήματα, καθώς και το κοινό `FIONA_PASSWORD`. Για διαφορετικό Basic Auth ανά λογαριασμό, χρησιμοποίησε τα αντίστοιχα `FIONA_<STORE>_BASIC_AUTH`; αλλιώς χρησιμοποιείται το κοινό `FIONA_BASIC_AUTH`. Επιβεβαίωσε με τη Fiona ποια ρύθμιση ισχύει για τους λογαριασμούς σου. Όρισε ξεχωριστό, ισχυρό `APP_PASSWORD`. Για διορθώσεις στο tracker, όρισε και το `PROJECT_TRACKER_SESSION_SECRET` και το `PROJECT_TRACKER_TEAM_PASSWORDS` ως JSON αντικείμενο με κλειδιά της μορφής `storeId|ακριβές teamName`. Το `.env` περιέχει μυστικά και δεν πρέπει να κοινοποιείται.

Για production-like εκτέλεση, κάνε build και ξεκίνησε τον backend:

```
npm run build:frontend
npm run build:tracker
npm run start:backend
```

Άνοιξε `http://localhost:<PORT>` για τη Fiona εφαρμογή ή `http://localhost:<PORT>/tracker/` για το tracker. Οι πόντοι αρχίζουν από την πρώτη ενεργοποίηση, όχι από παλαιότερες εργασίες.

Για ανάπτυξη, άνοιξε δύο terminals στο project root και εκτέλεσε:

```
npm run dev:backend
npm run dev:frontend
npm run dev:tracker
```

Το κύριο app είναι στο `http://localhost:5173/app/`. Το tracker development server είναι στο `http://localhost:5174/tracker/` και κάνει proxy τα API requests στο `http://localhost:4000` από προεπιλογή. Αν ο backend χρησιμοποιεί άλλη θύρα, όρισε `BACKEND_URL` πριν ξεκινήσεις το `dev:tracker`.

Για τα tests του backend:

```
npm --prefix backend test
```

Πρόσβαση μέσω Cloudflare Tunnel

Το Tunnel εκθέτει την εφαρμογή του υπολογιστή μέσω HTTPS. Ο υπολογιστής, ο backend και το Tunnel πρέπει να παραμένουν σε λειτουργία και ο υπολογιστής να μην μπει σε αναστολή. Τα Quick Tunnels είναι προσωρινά και η διεύθυνση αλλάζει σε κάθε εκκίνηση· είναι κατάλληλα για demo, όχι για μόνιμη υπηρεσία.

Προσωρινό Quick Tunnel

1. Ρύθμισε το `backend/.env`, κάνε build και ξεκίνησε τον backend από το project root:

`powershell npm run build:frontend npm run build:tracker npm run start:backend`

2. Επιβεβαίωσε ότι το `/` ανακατευθύνει στο κύριο app (`/app/`) και ότι το tracker ανοίγει στο `/tracker/`, για παράδειγμα στο `http://localhost:4001` αν έχεις ορίσει `PORT=4001`.
3. Σε δεύτερο PowerShell, εγκατάστησε το `cloudflared` αν δεν υπάρχει ήδη:

`powershell winget install --id Cloudflare.cloudflared --exact`

Αν το `winget` δεν είναι διαθέσιμο, ακολούθησε τις επίσημες οδηγίες εγκατάστασης της Cloudflare για Windows. 4. Ξεκίνα το Tunnel χρησιμοποιώντας τη θύρα του backend:

`powershell cloudflared tunnel --url http://localhost:4001`

5. Άνοιξε στο κινητό τη διεύθυνση `https://....trycloudflare.com/app/` για το κύριο app ή `https://....trycloudflare.com/tracker/` για το tracker. Το σκέτο domain ανακατευθύνει στο `/app/`. Μοίρασε τη διεύθυνση και το `APP_PASSWORD` μόνο με τους προβλεπόμενους χρήστες και μέσω χωριστού, έμπιστου καναλιού.
6. Άφησε ανοιχτά τα δύο terminals και τον υπολογιστή ενεργό. Όταν τελειώσεις, σταμάτησε το Tunnel και τον backend με `Ctrl+C` στα αντίστοιχα terminals.

API

- `GET /api/stores`: ονόματα και ρυθμισμένη κατάσταση καταστημάτων.
- `GET /api/tickets?store=STORE_ID`: snapshot καταστήματος. `store=all` επιστρέφει ενιαία λίστα με `storeId` σε κάθε ticket.
- `GET /api/closed-tickets/stats?month=YYYY-MM&store=STORE_ID`: μηνιαία λίστα και στατιστικά· `store=all` συγκεντρώνει όλα τα καταστήματα.
- `GET /api/closed-tickets/:orderId?store=STORE_ID`: αναλυτικά στοιχεία ticket. Το store είναι απαραίτητο για να επιλυθούν πιθανές επικαλύψεις IDs.
- `POST /api/login`, `POST /api/logout`, `GET /api/session`: σύνδεση και έλεγχος session. Τα προστατευμένα API απαιτούν signed `HttpOnly` session cookie.

Το `GET /api/tickets` επιστρέφει `{ status, data: { tickets, lastUpdatedAt, lastAttemptAt, totalTickets, error } }`. Η κανονικοποιημένη εγγραφή ticket περιλαμβάνει `customer`, `order`, `priority`, `sla`, `communication`, `dispatch` και `meta`.

Συγχρονισμός ολοκληρωμένων tickets

Το backend CLI ανακτά workorders από τη Fiona για συμπεριληπτικό εύρος ημερομηνιών και αποθηκεύει νέα ολοκληρωμένα tickets στη SQLite:

```
cd backend
npm run sync:closed -- --from-date 01.08.2026 --to-date 30.09.2026
```

Πρόσθεσε `--store komotini` ή `--store orestiada` για ιστορικό συγχρονισμό του αντίστοιχου λογαριασμού. Χωρίς `--store`, χρησιμοποιείται η Αλεξανδρούπολη. Η Fiona επιστρέφει τα ίδια workorders σε όλους τους λογαριασμούς (κοινό group `395`), γι' αυτό το CLI κρατά μόνο orders όπου το `serviceGroup` των λεπτομερειών περιέχει τη λέξη του καταστήματος (`ALEXANDROUPOL`, `KOMOTINI`, `ORESTIADA`). Η λέξη αλλάζει με `FIONA_<STORE>_SERVICE_GROUP_MATCH`. Η σύνοψη αναφέρει τα orders άλλων καταστημάτων στο `otherStore`.

Για εξαγωγή των εγγραφών του εύρους σε JSON, πρόσθεσε `--export-json` με τη διαδρομή του αρχείου:

```
npm run sync:closed -- --from-date 01.08.2026 --to-date 30.09.2026 --store komotini --export-json exports/komotini-tickets.json
```

Το αρχείο περιλαμβάνει τα δεδομένα που έχουν αποθηκευτεί στη βάση για το επιλεγμένο κατάστημα και το συμπεριληπτικό εύρος ημερομηνιών, καθώς και μεταδεδομένα καταστήματος, ημερομηνιών και πλήθους εγγραφών. Αν το αρχείο υπάρχει ήδη, αντικαθίσταται. Περιέχει προσωπικά δεδομένα πελατών· φύλαξέ το και κοινοποίησέ το με την ίδια προσοχή όπως τη βάση SQLite.

Για επανάκτηση και διόρθωση υπαρχουσών εγγραφών:

```
npm run sync:closed -- --from-date 01.08.2026 --to-date 30.09.2026 --refresh-existing
```

Το εύρος ζητείται από τη Fiona μία εργάσιμη ημέρα κάθε φορά. Η προεπιλεγμένη ομάδα υπηρεσίας είναι `395`. Για αλλαγή, πρόσθεσε `--service-group ID` μία ή περισσότερες φορές. Το CLI παραλείπει `orderId` που υπάρχουν ήδη στη βάση και αποθηκεύει μόνο εγγραφές με `orderStatus=COMPLETED` και έγκυρο `workOrderFinishedOnPremiseTime`. Δεν εισάγει το υπάρχον JSON export.

Από προεπιλογή, οι υπάρχουσες εγγραφές παραλείπονται. Με `--refresh-existing`, ενημερώνονται μόνο εγγραφές του εύρους για τις οποίες η Fiona επιστρέφει ολοκληρωμένο order, έγκυρη ώρα ολοκλήρωσης και επιτυχημένα όλα τα enrichment requests. Η επιλογή δεν εισάγει εγγραφές που λείπουν· αποτυχημένες ή ελλιπείς απαντήσεις αφήνουν την υπάρχουσα εγγραφή αμετάβλητη. Η σύνοψη αναφέρει χωριστά τις ενημερώσεις και τις εισαγωγές.

Το CLI εμφανίζει τη διαδρομή του script, το εύρος ημερομηνιών, τις ομάδες υπηρεσίας, τη βάση, την πρόοδο ανά ημέρα και στάδιο αιτήματος, καθώς και την τελική σύνοψη. Δεν εμφανίζει ονόματα πελατών, τηλέφωνα ή Α.Φ.Μ. Τα delays των Fiona requests είναι από προεπιλογή 1–2,5 δευτερόλεπτα και ρυθμίζονται με `FIONA_REQUEST_MIN_DELAY_MS` και `FIONA_REQUEST_MAX_DELAY_MS`.

Η βάση βρίσκεται από προεπιλογή στο `backend/data/closed-tickets.sqlite`. Για άλλη τοποθεσία όρισε `CLOSED_TICKETS_DB_PATH`.

SLA και συμπεριφορά δεδομένων

Για αναλυτικούς κανόνες, τύπους υπολογισμού και συγκεντρωτικά ευρήματα από τα συγχρονισμένα δεδομένα, δες το SLA.md.

- Το `workOrderSlaDate` είναι ο στόχος ολοκλήρωσης και συγκρίνεται με το `workOrderFinishedOnPremiseTime`. Αν λείπει ημερομηνία, το αποτέλεσμα είναι `unknown`.
- Το order `deadline` είναι η προθεσμία πρώτης κλήσης (παράθυρο 4 ωρών), αποθηκεύεται ως `call_deadline` και δεν αξιολογείται έναντι της ολοκλήρωσης.
- Κατά το polling, αποθηκεύονται οι καταστάσεις ανοιχτού ticket στον `ticket_status_observations`. Η πρώτη καταγεγραμμένη κατάσταση `CALL LATER` ή `APPOINTMENT` συγκρίνεται με το call deadline. Αν λείπει η κατάσταση ή η προθεσμία, το αποτέλεσμα πρώτης κλήσης είναι `unknown`.
- Η Fiona επιστρέφει πάντα `N` στο `slaSurpassed`, επομένως αυτό το πεδίο δεν αποθηκεύεται.
- Αν η Fiona δεν είναι διαθέσιμη, ο backend διατηρεί το τελευταίο επιτυχημένο snapshot ως `stale`. Demo data χρησιμοποιούνται μόνο όταν δεν υπάρχει διαθέσιμο live snapshot. Ταυτόχρονες αιτήσεις refresh μοιράζονται το ίδιο Fiona request.

Ασφάλεια και ιδιωτικότητα

Το `backend/.env` αγνοείται από το Git. Μην αποθηκεύεις πραγματικά secrets σε source control, screenshots ή chat. Χρησιμοποίησε ισχυρό, μοναδικό `APP_PASSWORD` και κράτησε ιδιωτικά τη βάση SQLite και τα backups: περιέχουν προσωπικά δεδομένα πελατών, όπως τηλέφωνα και Α.Φ.Μ. Πριν από αλλαγή schema, πάρε backup της βάσης. Αν credentials κοινοποιηθούν κατά λάθος, άλλαξέ τα. Πριν εκθέσεις την εφαρμογή σε δημόσιο δίκτυο, περιόρισε την πρόσβαση στους εξουσιοδοτημένους χρήστες. Το Cloudflare Tunnel δεν αντικαθιστά το login της εφαρμογής.

Η εφαρμογή εγκαθίσταται ως PWA μέσω του manifest και του service worker.